Multi-Document RAG Chatbot

Case Study

Flowise · Groq LLM · Google Gemini Embeddings

Watch demo video | linkedin.com/in/salonee-s-201564399

OVERVIEW

Organizations and individuals often need to search across several long documents at once — policy papers, strategy reports, internal manuals — where a simple keyword search falls short. This project builds a retrieval-augmented generation (RAG) chatbot that lets a user upload multiple PDF documents and ask natural-language questions, with the system returning grounded, source-cited answers rather than generic responses. It was built end-to-end using Flowise, with Google Gemini for embeddings and Groq for fast LLM inference.

PROBLEM

- Long-form documents (policy papers, strategy PDFs) are hard to search for specific answers using Ctrl+F or manual skimming
- Generic LLM chat tools can hallucinate or answer from general knowledge instead of the actual source document
- Most existing tools handle a single document at a time, not a combined knowledge base across several files

APPROACH & ARCHITECTURE

1. Document Ingestion

PDF files are uploaded and split into smaller chunks using a Recursive Character Text Splitter (chunk size 1500, overlap 300), which preserves context across chunk boundaries while keeping each piece small enough for accurate retrieval.

2. Embedding & Storage

Each chunk is converted into a vector embedding using Google's Gemini Embedding model (gemini-embedding-001), then stored in an in-memory vector store configured to retrieve the top 10 most relevant chunks per query.

3. Retrieval & Generation

When a user asks a question, the system retrieves the most relevant chunks and passes them, along with the question, to a Groq-hosted LLM (openai/gpt-oss-20b) for fast, low-latency response generation.

4. Conversation Memory

A buffer memory component tracks conversation history, so follow-up questions are understood in context rather than treated as isolated queries.

5. Orchestration

All components are tied together using a Conversational Retrieval QA Chain, configured to return source documents alongside each answer — so responses can be traced back to the original PDF, improving trust and auditability.

TECH STACK

- Flowise (no-code AI orchestration)
- Google Gemini Embedding API
- Groq (LLM inference — GPT-OSS-20B)
- In-memory vector store
- Conversational Retrieval QA Chain with buffer memory

KEY DESIGN DECISIONS

- Chose Groq for inference to minimize response latency, which matters for a chat-style interface
- Enabled source document return on every response, prioritizing traceability and reducing the risk of unverifiable answers

- Used a moderate chunk size (1500 characters) with overlap to balance retrieval precision against losing context across chunk splits

OUTCOME

The result is a working chatbot that can answer questions across multiple policy documents (demonstrated using India's National Strategy for Artificial Intelligence and the NITI Aayog AI white paper) with source-grounded responses and conversational memory. The project moved from concept to a functioning, demoable build, including a recorded walkthrough of the architecture and a live Q&A example.

WHAT I'D IMPROVE NEXT

- Move from an in-memory vector store to a persistent vector database for production use
- Add evaluation metrics to measure retrieval accuracy and answer relevance over time
- Support additional file types beyond PDF (Word docs, web pages)